

ENERGY CONSUMPTION FORECASTING

Product Requirements Document



ENERGY CONSUMPTION FORECASTING

Product Requirements Document

Machine Learning–Powered Energy Consumption Prediction System

Project	Energyforecasting
Document Type	Product Requirements Document (PRD)
Technology Stack	Python • Flask • XGBoost • NumPy • Joblib • HTML/CSS/JavaScript
Deployment	Render
Document Version	1.0

Submission-ready technical specification

Prepared for project evaluation, technical presentation, and portfolio documentation.

Table of Contents

Section	Title	Page
1	Executive Summary	3
2	Problem Statement	3
3	Product Vision	4
4	Objectives	4
5	Target Users	5
6	Product Scope	5
7	Key Features	6
8	User Flow	6
9	Functional Requirements	7
10	API Specification	8
11	Machine Learning Component	9
12	Feature Definition	9
13	System Architecture	10
14	Technology Stack	10
15	Project Structure	11
16	Non-Functional Requirements	12
17	Error Handling	12
18	Security Considerations	13
19	Deployment Requirements	13
20	Current Limitations	14
21	Future Enhancements	14
22	Success Metrics	15
23	Acceptance Criteria	15

24	Risks and Mitigation	16
25	Deliverables	16
26	Conclusion	17
27	Appendices	17

1. Executive Summary

Energy Consumption Forecasting is a web-based machine learning application designed to predict electricity or energy consumption from temporal and historical consumption features. The system combines a Flask backend, a tuned XGBoost model, a browser-based interface, and REST-style API endpoints to provide an accessible prediction workflow for students, evaluators, developers, and future energy-management applications.

The implemented prediction flow accepts time-related and historical consumption inputs, validates the request, loads the trained model artifacts from the `models/` directory, generates a prediction through XGBoost, and returns the result as JSON. The actual machine-learning prediction endpoint is `/api/predict`. The application also exposes hourly and weekly forecast endpoints for visualization; however, their current values are predefined/static and are not dynamically generated by XGBoost.

Implementation status. The current release implements Flask-based inference, input validation, model metadata access, a web interface, and deployment configuration. Dynamic hourly and weekly forecasting, retraining, authentication, database persistence, and model monitoring remain future functionality.

2. Problem Statement

Energy consumption changes according to time of day, day of week, weekend behavior, seasonal or monthly patterns, recent consumption, and recurring daily or weekly demand. Without a structured forecasting approach, users may have difficulty understanding these patterns or producing consistent estimates for planning and analysis.

Forecasting can support energy planning, demand awareness, resource management, future energy optimization, and smart-energy applications. A lightweight prediction service can make engineered temporal and historical features available through a simple interface while providing a foundation for later integration with real-time meters, databases, monitoring systems, and advanced energy-management workflows.

3. Product Vision

Energy Consumption Forecasting will provide a lightweight and accessible machine-learning-powered platform that transforms historical and temporal energy information into understandable predictions through a web interface and an API. The product is intended to demonstrate a practical path from feature engineering and model inference to cloud-

deployed application delivery while preserving a clear boundary between implemented capabilities and future enhancements.

4. Objectives

- **Machine-learning prediction** — Provide energy-consumption predictions using the trained XGBoost model.
- **Accessible interaction** — Provide a simple web interface for entering prediction features and viewing results.
- **API exposure** — Expose the model through a documented REST-style API for external applications.
- **Input quality** — Validate required fields and allowed value ranges before inference.
- **Structured output** — Return predictions and errors in JSON format.
- **Supporting services** — Provide model information, hourly forecast data, and weekly forecast data through separate endpoints.
- **Cloud readiness** — Support deployment on Render using a production WSGI server.
- **Future foundation** — Establish an application structure that can later support dynamic forecasting, persistence, monitoring, and integrations.

5. Target Users

User Group	Primary Need
Students	Understand and demonstrate an end-to-end machine learning web application.
Project evaluators	Review implementation scope, architecture, APIs, and technical decisions.
Hackathon judges	Assess the usefulness, clarity, and deployability of the solution.
Developers	Consume prediction endpoints or extend the application.
ML learners	Study feature-based inference, model artifacts, and API integration.
Future users	Support energy-management systems, smart-grid applications, energy analysts, and organizations monitoring demand.

6. Product Scope

In Scope	Out of Scope
Flask web application	Automatic model retraining
XGBoost model inference	Real-time smart-meter integration
Input validation and structured errors	User authentication
Prediction and model-information APIs	Database-backed prediction history
Hourly and weekly forecast endpoints	Automated dynamic hourly or weekly forecasting
Web user interface	Production model monitoring
Render deployment with Gunicorn	Model drift detection

7. Key Features

Feature	Description
Energy Prediction	Uses trained XGBoost model to predict energy consumption
Feature Validation	Validates required fields and allowed ranges
Model Information	Returns model name and feature names
Forecast Data	Provides hourly and weekly forecast data
Web Interface	Provides an accessible frontend
API Integration	Allows external applications to consume predictions
Cloud Deployment	Supports deployment using Render

8. User Flow

User → Web Interface → Input Energy/Time Features → Flask API → Input Validation → XGBoost Model → Prediction → JSON Response → Frontend Result

- Enter features** — The user enters temporal and historical consumption features through the web interface.
- Submit request** — The frontend sends the values to the Flask prediction API.
- Validate input** — Required fields and permitted ranges are checked before model inference.
- Generate prediction** — The XGBoost model receives the validated feature vector and produces an energy-consumption prediction.
- Render result** — The API returns JSON and the frontend displays the returned result to the user.

9. Functional Requirements

ID	Requirement
FR-01	The system shall accept prediction requests through <code>/api/predict</code> .
FR-02	The request shall contain Hour, Day, Month, Day_of_Week, Is_Weekend, Lag_1, Lag_24, Lag_168, Rolling_3, Rolling_24, and Rolling_168.
FR-03	The system shall validate Hour 0–23, Day 1–31, Month 1–12, Day_of_Week 0–6, and Is_Weekend 0 or 1. Missing fields shall produce structured errors.
FR-04	<code>/api/model-info</code> shall return the model name and feature names.
FR-05	<code>/api/forecast</code> shall return hourly forecast data.
FR-06	<code>/api/weekly-forecast</code> shall return weekly forecast data.

10. API Specification

Endpoint	Method	Purpose
<code>/</code>	GET	Loads main web page
<code>/api/model-info</code>	GET	Returns model information
<code>/api/predict</code>	POST	Generates ML prediction
<code>/api/forecast</code>	GET	Returns hourly forecast
<code>/api/weekly-forecast</code>	GET	Returns weekly forecast

Example request — POST `/api/predict`

```
{
  "Hour": 14,
  "Day": 15,
  "Month": 6,
  "Day_of_Week": 2,
  "Is_Weekend": 0,
  "Lag_1": 2.30,
  "Lag_24": 2.40,
  "Lag_168": 2.50,
  "Rolling_3": 2.35,
  "Rolling_24": 2.42,
  "Rolling_168": 2.48
}
```

Successful response

```
{
  "success": true,
  "prediction": 2.45
}
```

Error response

```
{
  "success": false,
  "error": "Missing values: Hour"
}
```

Invalid requests return structured JSON and HTTP 400 where applicable. The API contract is intentionally simple so that the frontend and external consumers can handle success and failure responses consistently.

11. Machine Learning Component

The current project uses a Tuned XGBoost model for energy-consumption prediction. The model receives engineered temporal and historical features and produces an energy-consumption prediction for the submitted feature vector. The actual ML prediction endpoint is `/api/predict`.

Artifact	Repository Path	Purpose
Model artifact	<code>models/energy_consumption_model.pkl</code>	Stores the trained Tuned XGBoost model.
Feature metadata	<code>models/feature_names.pkl</code>	Stores the feature names and expected ordering metadata.
Scaler artifact	<code>models/scaler.pkl</code>	Exists in the repository but is not currently applied during Flask inference.

Critical implementation status. Although `scaler.pkl` exists in the repository, the current Flask inference implementation does not currently apply the scaler before prediction. This document does not claim that scaling is used.

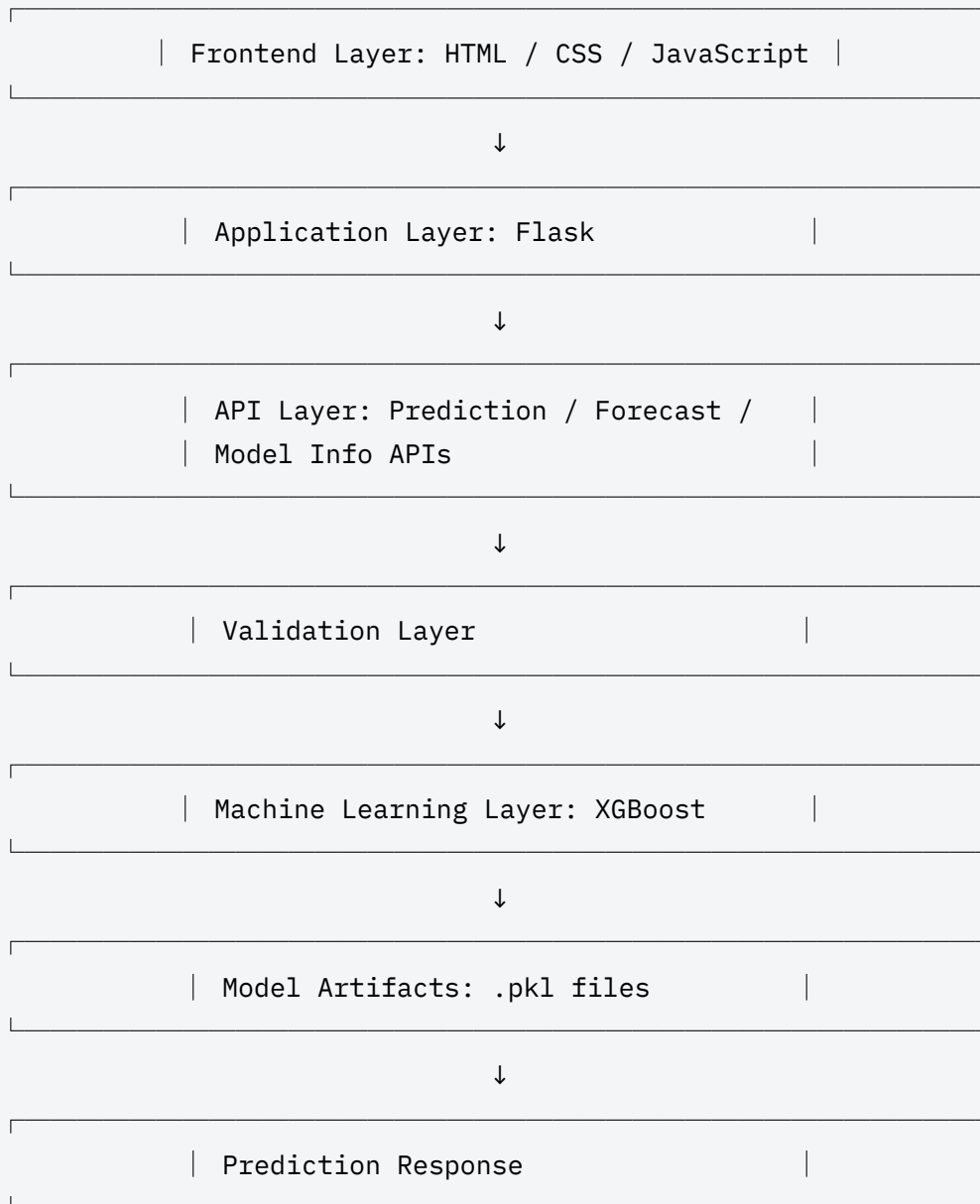
12. Feature Definition

Feature	Description
Hour	Hour of day
Day	Day of month
Month	Month of year
Day_of_Week	Weekday index
Is_Weekend	Weekend indicator
Lag_1	Recent previous consumption
Lag_24	Approximately previous-day consumption
Lag_168	Approximately previous-week consumption
Rolling_3	Short rolling statistic
Rolling_24	Daily-window rolling statistic
Rolling_168	Weekly-window rolling statistic

Lag features provide recent, approximately daily, and approximately weekly reference points. Rolling features summarize short, daily-window, and weekly-window behavior, helping the model capture local trends and recurring consumption patterns alongside calendar features.

13. System Architecture

-



- **Frontend layer** — Presents input controls and renders returned results in the browser.
- **Application layer** — Hosts routes, request handling, and coordination through Flask.
- **API layer** — Exposes prediction, forecast, and model-information services.
- **Validation layer** — Checks required fields and permitted ranges before inference.
- **Machine learning layer** — Loads and invokes the Tuned XGBoost model for the prediction endpoint.
- **Artifact layer** — Provides the serialized model and metadata stored as .pk1 files.
- **Response layer** — Returns a structured JSON prediction or error response.

The forecast endpoints are available for visualization but are currently backed by predefined/static forecast values rather than dynamically generated XGBoost predictions.

14. Technology Stack

Technology	Role
Python	Backend and ML environment
Flask	Web framework
Flask-CORS	Cross-origin API support
XGBoost	Machine learning model
NumPy	Numerical processing
Joblib	Model loading
HTML	Structure
CSS	Styling
JavaScript	Frontend interaction
Gunicorn	Production server
Render	Cloud deployment

15. Project Structure

-

```
Energyforecasting/  
|  
|—— models/  
|   |—— energy_consumption_model.pkl  
|   |—— feature_names.pkl  
|   |—— scaler.pkl  
|  
|—— static/  
|  
|—— templates/  
|  
|—— app.py  
|—— requirements.txt  
|—— render.yaml
```

- **models/** — Contains the serialized model, feature metadata, and scaler artifact.

- **static/** — Holds static frontend assets such as stylesheets and scripts.
- **templates/** — Holds HTML templates rendered by the Flask application.
- **app.py** — Defines the Flask application and API routes.
- **requirements.txt** — Declares Python dependencies required to run the project.
- **render.yaml** — Provides Render deployment configuration.

16. Non-Functional Requirements

- **Performance** — Prediction requests should complete quickly for normal single-record inference.
- **Availability** — The deployed application should be accessible through its public URL.
- **Maintainability** — Frontend, backend, and model artifacts should remain organized.
- **Scalability** — The API architecture should allow future database and external integrations.
- **Usability** — Errors should be understandable to users and API consumers.
- **Portability** — Dependencies should work in a Linux cloud deployment environment.

17. Error Handling

The application shall validate requests before model inference and return structured JSON responses for invalid or failed operations. Invalid requests shall return HTTP 400 where appropriate, allowing the frontend and external clients to distinguish input errors from successful predictions.

Condition	Expected Handling
Missing values	Identify missing required fields and return a structured error such as Missing values: Hour.
Invalid hour	Reject values outside 0–23.
Invalid day	Reject values outside 1–31.
Invalid month	Reject values outside 1–12.
Invalid weekday	Reject values outside 0–6.
Invalid weekend flag	Reject values other than 0 or 1.
Model prediction errors	Return a structured failure response when inference cannot be completed.
Unexpected exceptions	Return a controlled JSON error without exposing sensitive implementation details.

18. Security Considerations

- **Secrets** — Do not expose secrets, credentials, or private configuration in source code or responses.
- **Input validation** — Validate all user-provided values before they reach the model or application logic.
- **Transport security** — Use HTTPS in production deployments.
- **Administrative access** — Protect any future administrative endpoints with authentication and authorization.
- **Execution safety** — Do not allow arbitrary file execution or arbitrary model loading from user input.
- **Artifact control** — Keep model artifacts controlled and deploy only trusted serialized files.

- **Configuration** — Use secure deployment configuration and avoid committing sensitive environment values.**19.**

Deployment Requirements

The application is intended for deployment on Render. The Flask application object is named `app`, and the project uses `render.yaml` for deployment configuration. Gunicorn is used as the production WSGI server.

Build Command:

```
pip install -r requirements.txt
```

Start Command:

```
gunicorn app:app --bind 0.0.0.0:$PORT
```

20. Current Limitations

Current implementation limitations. The following limitations are part of the current state and must be considered when evaluating the application.

1. `/api/forecast` currently returns predefined/static forecast values.
2. `/api/weekly-forecast` currently returns predefined/static forecast values.
3. These endpoints are not currently generating dynamic predictions using XGBoost.
4. `/api/predict` is the actual ML prediction endpoint.
5. The scaler artifact exists but is not currently used in Flask inference.
6. There is no database.
7. There is no authentication.
8. There is no automated retraining.
9. There is no model monitoring.

21. Future Enhancements

- Dynamic hourly forecasting

- Dynamic weekly forecasting
- Recursive multi-step forecasting
- Consistent scaler integration
- Actual versus predicted charts
- Model evaluation dashboard
- Prediction history
- Database integration
- User authentication
- Automated model retraining
- Model drift detection
- SHAP-based explainability
- Peak consumption alerts
- Energy-saving recommendations
- Smart-meter integration

22. Success Metrics

Indicator	Target Outcome	How Evaluated
Successful model prediction	Valid prediction returned for valid feature data.	Submit a valid request to <code>/api/predict</code> and inspect the JSON response.
Valid API responses	Endpoints return documented response structures.	Exercise the documented GET and POST endpoints.
Proper error handling	Invalid requests are rejected with understandable structured errors.	Test missing fields and invalid ranges.
Successful deployment	Application runs through the configured cloud deployment.	Deploy using Render and access the public URL.
Usable frontend	Users can submit features and understand returned results.	Perform an end-to-end browser workflow.
Maintainable architecture	Frontend, API, validation, and model responsibilities remain separable.	Review project structure and route organization.
Future integration capability	Architecture can support later model and data integrations.	Review documented extension points and scope boundaries.

Measurement boundary. No accuracy values or performance statistics are claimed because none were provided for this project.

23. Acceptance Criteria

Acceptance Item	Verification Description
Homepage loads successfully.	Open the application root URL and confirm that the main web page is rendered.
Model artifacts load successfully.	Start the application and confirm that the required model artifacts are available to the inference process.
<code>/api/model-info</code> returns correct metadata.	Send a GET request and verify the model name and feature names.
<code>/api/predict</code> returns a prediction for valid data.	Submit all required features with valid values and verify a successful JSON prediction.
Missing fields generate appropriate errors.	Omit a required field and verify a structured error response with HTTP 400 where applicable.
Invalid ranges are rejected.	Submit invalid temporal or weekend values and verify that validation prevents inference.
Forecast endpoints return valid JSON.	Send GET requests to the hourly and weekly forecast endpoints and verify valid JSON output.
Application can run using Gunicorn.	Run the documented start command and confirm that the Flask application is served.
Application can be deployed on Render.	Deploy using the project dependencies, start command, and <code>render.yaml</code> configuration.

24. Risks and Mitigation

Risk	Mitigation
Model artifact incompatibility	Use compatible package/Python versions
Incorrect feature order	Maintain feature-name list and training order
Data distribution changes	Monitor and retrain
Static forecasts misunderstood as ML forecasts	Clearly document current implementation
Large dependency build time	Keep requirements minimal

25. Deliverables

Deliverable	Purpose
Flask application	Backend routes, validation, web serving, and prediction orchestration.
XGBoost model	Serialized trained model used for current ML inference.
Feature metadata	Feature names and expected inference ordering.
Frontend	Browser interface for entering features and viewing results.
Requirements file	Python dependency declaration for local and cloud execution.
Render configuration	Deployment configuration for the Render environment.
Product Requirements Document	Formal specification of scope, requirements, architecture, and evaluation criteria.

26. Conclusion

Energy Consumption Forecasting provides a practical machine-learning foundation for energy-demand prediction. It combines a tuned XGBoost model, engineered temporal and historical features, Flask APIs, a browser interface, validation, and Render deployment into a coherent technical project suitable for academic submission and demonstration. The documented separation between the implemented prediction endpoint and the current static forecast endpoints preserves technical accuracy while establishing a clear path toward a more advanced energy-management platform.

27. Appendices

Appendix A — API Response Patterns

Successful Prediction

```
{
  "success": true,
  "prediction": 2.45
}
```

Error

```
{
  "success": false,
  "error": "Error message"
}
```

Appendix B — Project Reference

Project Repository: Energyforecasting

GitHub: [https://github.com/shruti55-del/Energyforecasting`](https://github.com/shruti55-del/Energyforecasting)